ChatGPT

Executive Summary

This report identifies several concrete, implementable technical inventions in the provided “Brudo” runtime architecture. We focus on mechanisms exposed by the code and terminal evidence, **not** on broad governance concepts. The strongest patent candidates are narrow improvements to state-admission and execution pipelines that improve system integrity. In particular, the **Typed State Admission Engine** and **Consequence-Preserving Transition Validator** emerged as *Tier-1* nuclei. We also identify related families (reconstitution firewall, typed predicate algebra, admission receipts, and consequence-identifiability gating) that enable or refine these core inventions. We recommend filing patents on the admission engine and transition validator (primary claims), with narrower follow-ups for predicate receipt systems; broader “constitutional” ideas should remain research guidance.

Concretely, the **state admission engine** is a specialized module that *validates* a reconstructed snapshot (candidate state) before accepting it as authoritative, using a fixed set of typed predicates (e.g. byte-identity, source existence, authority scope, etc.) with fail-closed semantics. This is analogous to—and distinct from—known admission controllers (e.g. in Kubernetes)【59†L47-L52】. The evidence shows the current system trusts the snapshot without this gate, and the proposed engine would implement this gate as code. Likewise, the **transition validator** enforces that every admitted distinction (difference in state) is assigned exactly one lawful *disposition* (action), closing the loop on state consistency. We also uncover a new **identifiability gate**: before applying a transition, the system derives the consequences on all compatible “worlds” and withholds execution unless all agree. This uses counterfactual tests (e.g. swap labels and re-derive consequences) to ensure no circular reasoning. All of these have clear computational definitions, algorithms, and data structures (see diagrams below).

Each candidate is examined in turn. For each we describe the **technical problem**, sketch the **minimal technical solution** (algorithms, data formats, state machines), point to **implementation evidence** in the codebase or transcripts, propose relevant **controls or tests**, survey **prior art** (e.g. runtime verification, active learning, Kubernetes admission, W3C-Provenance, In-toto, MPEP guidance), note how our solution differs, and assess **enablement gaps** and patent risk. We also draft suggested claim language and list experiments needed to validate these concepts.

Priority and Recommendations: The Typed State Admission Engine and Transition Validator rank highest (file candidates). The admission pipeline and predicate algebra (including receipts) are closely related and likely support claims (consider provisional filings). The consequence-identifiability machinery is novel but subtle; it may warrant a separate provisional once initial prototypes exist. We **do not** recommend independent claims on broad “constitutional runtime” or “meaning generation” metaphors at this stage, as these are abstract.

The table below summarizes the candidates (⭐=strength) and recommended actions. (⭐=file candidate, ★★=develop further, ☆=research, etc.)

Candidate	Novelty Strength	Implementation Support	Prior-Art Risk	Enablement Gap	Recommendation
Typed State Admission Engine	⭐⭐⭐⭐⭐	Partial (sketches exist)	Medium (container admission) 【59†L47-L52】	Need code & receipts	**FILE_CANDIDATE**
Consequence-Preserving Transition Validator	⭐⭐⭐⭐⭐	Partial (conceptual)	Medium (runtime assertions) 【10†L74-L82】 【59†L47-L52】	Needs spec & data types	**FILE_CANDIDATE**
Candidate→Authoritative Admission Pipeline	⭐⭐⭐⭐☆	Partial (architecture)	Medium (verify-gated control 【12†L368-L372】)	Implementation path	**FILE_CANDIDATE**
Admission Predicate Algebra (types/receipt)	⭐⭐⭐⭐☆	Minimal (structure known)	Medium (predicate logic, in-toto proofs?)	Formalization needed	**FILE_CANDIDATE**
Reconstitution Firewall	⭐⭐⭐⭐☆	Conceptual (suggested)	High (checkpoint/ consistency)	Need concrete design	**PROVISIONAL**
Typed Admission Receipts	⭐⭐⭐☆☆	Minimal (log-like)	Medium (audit logging)	Formal spec + format	**MERGE/HOLD**
Consequence-Identifiability Gate	⭐⭐⭐☆☆	Absent (not coded)	High (known LTL monitors 【24†L13-L20】)	Prototype needed	**PROVISIONAL**
Counterfactual Independence Assay	⭐⭐⭐☆☆	Absent	High (counterfactual literature)	Prototype needed	**PROVISIONAL**
Minimum Observation Engines	⭐⭐☆☆☆	Absent	High (active learning) 【27†L49-L57】	Implementation needed	**HOLD**

Candidate	Novelty Strength	Implementation Support	Prior-Art Risk	Enablement Gap	Recommendation
False-Withholding Detection	⭐⭐☆☆☆	Absent	Medium (fail-safe logic)	Formal spec	**HOLD**
Result-Contract / Gen-Exec Boundary	⭐⭐⭐☆☆	Partial (observed bug)	High (schema validation known 【68†L126-L134】)	Need novel binding	**MERGE/HOLD**

【45†embed_image】 *Fig. 1: Example admission pipeline architecture. Execution-produced work (execution plane) passes through a verify gate (verification plane) enforcing typed predicates before entering the authoritative state (context plane). Dashed arrows indicate governance constraints (from [12†L364-L372]).*

1. Typed State Admission Engine

Technical problem: In the current system, a "reconstructed" snapshot (e.g. from logs or history) is trusted wholesale once loaded. This can allow stale, unauthorized, or out-of-date information to propagate. We need a gate that checks *every* proposed state before trusting it for execution (preventing unvetted state from entering the runtime).

Solution (inventive concept): A *state admission engine* that takes an **Observed Candidate State** and applies a fixed set of **typed admission predicates** before converting it into the **Authoritative Runtime State**. These predicates include (e.g.) **BYTE_IDENTITY** (verify content hash consistency), **SOURCE_EXISTENCE** (verify origin metadata), **TRACKED_STATE** membership, **TEMPORAL_SCOPE** (timestamp validity), **AUTHORITY_SCOPE** (signature or policy checks), **NON_INTERFERENCE** (no conflicting writes), **CANON_ELIGIBILITY** (the state's placement in the canon) etc. Only if *all* mandatory predicates succeed (fail-closed: any failure aborts admission) is the candidate state accepted. Internally, each predicate yields a typed receipt object with evidence (e.g. which bytes matched a hash, which keys were present, etc.), similar to a structured audit log【59†L47-L52】. After admission, the system seals the state (e.g. by writing an "admission packet" or receipt) linking it to the job identity.

Data structures/algorithm: Represent the state as packets or objects. Admission is a deterministic procedure: 1. Compute *CandidateState* from snapshot (e.g. parse JSONL).
2. For each predicate φ_i: evaluate φ_i(CandidateState) → {PASS, FAIL}; record evidence in a **AdmissionReceipt**.
3. If $\wedge\varphi_i$ = true (all passed), emit **AuthoritativeState**, else abort with failure receipt.
Receipts contain fields: predicate name, evaluated data, evidence artifacts (hashes, signatures, etc.), and Boolean result. The engine runs in O(n) on the state size for hash/scan predicates.

Implementation support: The arXiv case study already envisions such gating. It specifies an admission predicate vector $\varphi_1 \ldots \varphi_n$ (Eq.2, p.33) and states "**Operationally, Eq.2 is fail-closed: until all mandatory predicates hold, the claim remains non-admitted**"【12†L354-L362】. The existing codebase explicitly tracks "byte identity" checks and identity-preserving behaviors but does *not* enforce an admission step. For example, a "state_reconstitution" routine sets state without further check. Our terminal transcript notes:

"the current ledger trusts reconstructed state rather than admitting it through [the admission gate]." This gap is not by oversight but by design; closing it is a genuine improvement (cf. "admission control for process extension"【59†L27-L36】【59†L47-L52】).

Negative controls/tests:

- Load a malicious snapshot (e.g. bytes tampered) and assert engine rejects it (predicate failure).
- Load a valid snapshot but with wrong **AUTHORITY_SCOPE** (signature mismatch) – engine should fail.
- Back-to-back loads (same data) should yield identical receipts.
- Vary authority or time parameters (simulate replay under different authority scopes); ensure admission is recomputed and can vary (replay-time admission).

Prior art: The general idea resembles Kubernetes Admission Controllers【59†L47-L52】 and the "process admission control" disclosure【59†L47-L52】. Winchester's article (Defensive Pub.) describes structural predicates on proposed extensions, sealing decisions and failing safe with predicate-specific feedback【59†L47-L52】. However, that is applied to business-process graphs, not low-level state bytes. Container orchestration systems allow extension of processes only after admission checks (which we cite as analogous)【59†L47-L52】. Our invention is distinct in *binding identity of state to typed semantic predicates*. Also related is the "verify-gated completion" architecture【12†L354-L362】, but that focused on work tasks, not raw state.

Distinguishing features: Key differences:

- We treat *reconstructed state* (read from persistence) as *candidate* requiring admission, not immediately authoritative. (No prior system insists on this intermediate binding.)
- Predicates are **typed** and **data-driven**, not just policy code: each has its own logic and evidence (beyond mere schema check).
- Fail-closed semantics ensure strong invariants. And the separation of byte-integrity from authority scope is explicit (previous work stops at byte consistency).

Enablement gaps: We must define each predicate's logic clearly (the patent disclosure must specify at least one implementation of each), along with data formats for receipts. The codebase has hints (functions verifying digest, authorization checks), but we need a complete state machine and data model. Proof-of-concept code or pseudo-code would strengthen it.

Claim-scope risks: This invention could be viewed as an abstract "data-validation pipeline," so claims must stress the specific technical improvement (concrete algorithms for combining cryptographic hashes, role-based checks, etc.), not just the abstract idea of "checking data before use." We must avoid claiming "prevention of unauthorized state" in abstract terms; focus on transformation steps and receipts (cf. Alice/Mayo improvements test【17†L166-L175】【18†L67-L74】).

Filing recommendation: File candidate as first independent claim. Dependent claims: specifics of predicate types (byte-identity, authority-scope, temporal-scope), fail-closed enforcement, separation of state vs. execution, etc.

Suggested independent claim (draft): "A computer-implemented method for runtime state admission comprising: receiving a candidate state snapshot; evaluating a plurality of typed admission predicates

(including at least predicate for byte-integrity and predicate for authority-scope) on the candidate state; and transforming the candidate state into an authoritative runtime state only if all mandatory predicates succeed, wherein each predicate evaluation emits a receipt containing evidence of the check and a Boolean result, and wherein failure of any predicate aborts admission (fail-closed).”

Experiments/tests: Implement a stub admission engine in Brudo: simulate loading known-good vs. altered snapshots; verify corresponding predicate receipts. Add unit tests for each predicate. Ensure indistinguishable snapshots with different authority (by manipulating metadata) produce *different* admission outcomes or receipts.

2. Consequence-Preserving Transition Validator

Technical problem: After the state is admitted, the system still must validate any *transition* (change) to ensure it conforms to rules. We require that *every* admitted distinction (difference between states) is given a single lawful *disposition* (an action taken or rationale). Currently, transitions might be allowed without systematic disposition, risking unhandled exceptions or hidden state drift.

Solution: A **transition validator** that operates on `(AuthoritativeState, ProposedTransition)`. It examines each difference (distinction) introduced by the transition and computes exactly one disposition from a fixed set (e.g. *represented, transformed, superseded, discharged,* etc.). If any admitted distinction is left without an assigned disposition, or if more than one disposition applies, the transition is blocked. Essentially, it enforces a conserved quantity: the set of differences is “closed” by dispositions.

Data structures/algorithm: Represent differences as a list of *Distinction* objects (for each changed tuple, record old/new, context). For each Distinction, compute a DispositionType via rules or signature (this could be algorithmic: e.g. if content changed with proof-of-equivalence, disposition=“equivalence-proved”). The algorithm loops:
1. For each admitted Distinction d in AuthoritativeState: find exactly one matching rule/disposition.
2. If none or >1 rules apply, emit a *TransitionFailureReceipt*.
3. Otherwise commit transition, attach *DispositionReceipts* (one per distinction) recording type and evidence.

This is a purely computational check. It may reuse similar predicate machinery, but now over differences. It enforces determinism: same input yields same output disposition.

Implementation support: The concept of “disposition” appears in the terminal text (e.g. `"represented, transformed, superseded"`). The architecture paper [12] shows a similar idea: each completion claim goes through verify, and has an unambiguous outcome (success or blocked) after passing all predicates 【12†L354-L362】. Though not explicit, it suggests every admitted element had its fate decided. We would need to implement a new code path: likely a stage after state admission but before applying effects.

Negative controls/tests:

- Create a bogus transition that changes a field but define no disposition rule → must be rejected.
- Test a transition where two rules could apply; ensure validator flags error.
- Apply a transition producing only one type of change (e.g. add a log entry) and see that it gets a single disposition.

Prior art: This resembles “transaction validation” in databases or blockchains (e.g. every move must be justified). However, our key novelty is one-to-one dispositions. In-vivo systems may do consistency checks, but rarely frame them as dispositions. Runtime assertion systems (e.g. behavioral type checkers) are related. The verify-gated completion study【12†L354-L362】 comes close: it expects that each claim either is admitted or goes through blocked-recovery. However, it does not formalize per-distinction binding. The concept of ensuring every change has a justification might echo “functional correctness” checks, but again not packaged as dispositions.

Distinguishing features: The validator is *consequence-preserving* in that it explicitly checks the “conservation of distinctions”: nothing appears or disappears unaccounted. This differs from a simple functional test or schema validation; it ties each admitted change to a semantic category. The phrase “exactly one disposition per admitted distinction” is concrete and narrow.

Enablement gaps: We need to define the set of dispositions and the matching rules. The user text lists examples; patents should select a workable subset and show how they’re computed. Also, show interplay with state admission (maybe receipts link to dispositions).

Claim-scope risk: Without careful drafting, a claim might sound like “validate every difference” which could be seen as abstract. We must tie it to a specific data-structure (distinction objects) and a specific one-of-n categories rule.

Filing recommendation: File candidate. It can depend on the admission engine claim (involving both state and transition).

Suggested claim: “A system comprising: a state-transition module that, given an admitted state and a candidate transition, identifies all distinctions introduced by the transition; and a validator that, for each distinction, computes exactly one disposition type from a predetermined set, wherein the transition is accepted only if every distinction has exactly one disposition.”

Experiments: Implement a dummy transition validator: for each field-change, assign a disposition (e.g. ‘updated’ or ‘eliminated’). Create transitions that leave a distinction without disposition to confirm rejection. Simulate chains of transitions and ensure dispositions accumulate.

3. Disposition Closure and Reconstitution Firewall

Technical problem: We must ensure *global* conservation of allowable state differences: the net effect of all transitions should match the net dispositions. In particular, when restoring state (reconstitution) we need to guarantee that only admissible distinctions entered.

Solution: We formalize the invariant: “the lawful disposition of *every* admitted distinction is preserved” across any state transform. Practically, this means at admission time we record dispositions for each difference, and on any **reconstitution** (loading/replaying state), an additional firewall check verifies these dispositions still make sense (e.g. no missing disposition). This could be a check that the admitted difference *counts* (object identity) match the dispositions in the receipts.

Minimal technical combination: This is closely related to the above two: ensure that the set of dispositions in the protocol equals the set of distinctions in state. The firewall is a process during state loading: it compares the saved AuthoritativeState (with dispositions) to the new CandidateState after admission, requiring closure.

Evidence (implementation): The user context distinguishes "conserved quantity is *not* objects but dispositions." This is more conceptual. In code, it would involve storing dispositions in receipts and replaying them. The current system does not enforce this at reconstitution.

Controls/tests:

- Try loading a committed state under modified predicate outcomes; ensure firewall detects mismatches.
- Corrupt a disposition in a saved state and confirm the firewall blocks.

Prior art: Similar to the admission engine (container admission) and transaction consistency checks. Might relate to blockchains (UTXO set closure), but not exactly.

Distinguishing features: This is not just "state equality" but an invariant on recorded evidences. It's largely a derivative of the admission architecture, but potentially claimable as an aspect or dependent.

Recommendation: Provisional/Dependent claim; likely fold under admission engine.

4. Typed Admission Pipeline (2-Stage Runtime Admission)

Technical problem: Many systems combine state loading and execution without separation. Here, the architecture demands an explicit two-stage pipeline: **Candidate State → (Admission) → Authoritative State → (Execution)**.

Solution: Claim the architecture of a runtime with two separate stages: first, *admission stage* that validates/reconstructs state (as above); second, *execution stage* that only sees the admitted state. This contrasts with "load state then run" pipelines.

Evidence: The transcripts emphasize this is a key distinction:

> "Instead of: Restore → Continue. You propose: Reconstitute → Candidate → Admission → Authority → Execution."

This exact pipeline can be diagrammed. (See Fig.1 at start.) The arXiv paper likewise separates "execution plane" and "verification plane"【12†L374-L382】【12†L438-L442】.

Prior art: Standard systems do not do this. The GDPR in Kubernetes comes to mind, but again not exactly as a two-stage gate. The Winchester pub【59†L47-L52】 explicitly designs "admission control mechanisms that intercept and validate… applied to business graphs." But we claim this is a specific *two-step pipeline in code with typed predicates*.

Recommendation: Include in admission-engine claims as system architecture. Perhaps **file** as dependent claim or split into its own small family.

5. Admission Predicate Algebra and Receipts

Technical problem: Once introduced, each predicate (byte-identity, etc.) has rich structure (evidence, failure modes, reopening rules). This suggests a family of inventions: organizing predicates as typed algebraic objects, and producing structured receipts that can drive reopening of claims or replays.

Solution: Define a data model where each predicate has:

- a **type** (from a fixed schema),
- an evaluation function,
- a *receipt record* including evidence and a recommended reopening rule if it fails,
- a replay rule specifying how to check it on re-admission.

The receipts form audit logs usable to recompute decisions. The algebra part is that predicates can compose (e.g. disjunction of scope checks) in a mathematically consistent way.

Evidence: The terminal mentions “typed failure classification, admission receipts, transition receipts, reopening propagation, tracked/untracked partitioning, authority/temporal scope” as dependent claims. The code likely already has basic logs but not a formal model.

Prior art: In-toto (secure supply chain) and W3C PROV (provenance) use structured statements (entities, activities, agents) to trace execution. We can cite W3C PROV as a general model【17†L166-L175】 and in-toto for actionable attestations. Also “authorization receipts” draft IETF (schrock) might mention structured receipts. But those target provenance, not specifically dynamic state admission.

Recommendation: Develop as support claims. Possibly **file** narrower sub-claims.

6. Replay-Time Re-Admission

Technical problem: A state admitted once under one context (time, authority) may become inadmissible later. We need to *re-run the admission check on replay* to prevent stale trust.

Solution: Define **replay-time re-admission**: whenever a past completed state is reloaded (e.g. in a new process or after a crash), the state admission engine re-evaluates predicates using current parameters. Thus admission is never permanently assumed.

Evidence: The text explicitly contrasts “accepted state” vs. “re-admit ourselves”; it says replay-time admission is a stronger computation. No prior code for it exists.

Prior art: Not known. Possibly similar to checkpoint validations in distributed systems, but here specifically dynamic. Could cite “no replay: the policy says no self-approval, no bypass” from draft Schrock (TLA+ model)【61†L14-L18】 (hint of similar idea).

Recommendation: Likely part of the main admission-engine claims.

7. Consequence-Indexed Identifiability Gate

Technical problem: After admission, not all execution branches (consequences) may be *determinably* safe under partial knowledge. Given an observed state (observation class), a proposed consequence C may not be clearly determined – multiple worlds fit the observation. We need to delay execution of C until it is *identifiable*: uniquely determined by the admissible set of worlds.

Solution: Upon observation O (the admitted partial state), compute the **equivalence class** E of all possible real states consistent with O. Independently derive the proposed consequence C for each state in E. If all these derived consequences are the same value c, then permit execution of "C=c"; otherwise inhibit it (withhold execution) and label the consequence as **NOT_IDENTIFIABLE**. We explicitly perform a dependency check: ensure the basis for C does not causally derive from O (counterfactual test). This is a *consequence-relative gate*: identifiability is checked per-consequence.

Evidence: The user analysis calls this out: define

```
E = {R | O(R)=o},  Γ_C(E)={C(R): R∈E},  admit iff |Γ_C(E)|=1.
```

They propose a type `IdentifiabilityReceipt(C,o)`. This goes far beyond simple "conclusion unknown"; it factors in causal independence.

Prior art: LTL runtime verification (Bauer et al.) uses {true/false/inconclusive} semantics for partial traces【24†L13-L20】. However, that is for *propositional properties*, not distinguishing multiple valid outcomes. Active learning/version-space literature does optimize observation choice【27†L49-L57】, but not in execution gating. The unique element is gating by the *consequence* and using independent derivations as counterfactual tests. Causal inference literature (Pearl) covers identifiability of causal effects under partial models, but not in this runtime context.

Recommendation: This is an intriguing machine but speculative. Mark as **provisional / research**. We may file dependent claims on "three-state consequence admission" but need a working prototype first.

8. Counterfactual Anti-Circularity Assay

Technical problem: Even if Γ_C(E) is singleton, we must ensure we didn't "cheat" by circular reasoning. The evidence basis for deriving C might itself depend on the proposed outcome L, causing a feedback loop.

Solution: A **swap/intervention test**: For two hypothetical labels L_a, L_b in the equivalence class, "intervene" by swapping any steps in the derivation that reference L_a vs L_b (counterfactual world) while keeping base context fixed. Re-derive C in both. If swapping changes the result, the basis for C was contaminated by L. Only if C remains invariant under the swap is *independence* confirmed. Formally: check that `derive(B,R_a)` and `derive(B,R_b)` yield the same results even when labels in B are swapped.

Evidence: The user description provides equations and suggests making `consequence_source_independent` a non-boolean receipt object containing all dependency info. The concept is similar to counterfactual fairness checks.

Prior art: Counterfactual analysis is common in ML fairness (label-swapping tests) and causality. However, its use as a runtime admission step is novel. Defensive Pub [59] does not describe this.

Recommendation: Novel but complex. **Hold** or provisional until shown critical.

9. Observation Selection Engines (MinMissing/MinSufficient)

Technical problem: When a consequence is not identifiable, we need to decide what new information to acquire. The system can choose queries (additional observations) to refine E. Conversely, when identifying that C is already identifiable, we could simplify by removing redundant observations.

Solution: Define two optimization problems:

- **Minimum Missing Observation:** Given E where Γ_C(E)>1, find the admissible query q (or set of observations) minimizing cost such that it splits E and maximally reduces Γ_C heterogeneity (akin to expected information gain per cost). Possibly use Bayesian experimental design methods.
- **Minimum Sufficient Observation:** Given E and C, find the smallest subset of current observations that still yields |Γ_C(E)|=1 (i.e. drop any extraneous info).

Prior art: These are active-learning / experimental-design problems【27†L49-L57】 (uncertainty sampling, value-of-information). Identifying needed sensors or tests is a known problem in knowledge acquisition.

Recommendation: Too broad now. **Hold** until after implementing identifiability core.

10. False-Withholding Detection

Technical problem: We must avoid erroneously blocking a transition when it is actually safe. Specifically, if Γ_C(E) is singleton but we withhold anyway (false negative), it's a bug.

Solution: After identifiability check, assert that WITHHOLD only happens when Γ_C(E)>1. If |Γ_C(E)|=1, flag **UNNECESSARY_WORLD_RESOLUTION** and proceed.

Prior art: This is essentially verifying the gate is sound and complete relative to identifiability. Analogous to "soundness" in runtime monitors.

Recommendation: Likely folded into identifiability work.

11. Result-Contract / Generative-Execution Boundary

Technical problem: In the generative AI pipeline, the system allowed **non-deterministic LLM outputs** to bypass semantic contracts. The problem found: a job with a declared JSON schema (9 keys) was marked as identity-bound, but under low capacity it returned truncated JSON and failed, at medium it returned valid 6-

key JSON (accepted), and at high capacity an 8-key JSON (accepted) – all against a 9-key spec. The system equated syntactic validity (any JSON) with *semantic admissibility*, leaking unearned results.

Solution: A **contract-bound execution boundary**. Key components:

- **Contract Identity Object:** Treat the declared result schema/contract as an identity-bearing entity (e.g. a hash of the JSON schema or prompt).
- **Binding:** Incorporate the contract's identity into the job's unique ID (job = hash(input, modelConfig, contractID, ...)).
- **Admission Test:** Upon generation, parse output; then strictly validate it against *the exact contract tied to the job identity*. If any predicate (schema, semantic) fails, do not accept the output. Emit a semantic admission receipt documenting the failure.
- **Admission Receipts:** On success, issue a receipt binding jobID, resultID, contractID, and stating which predicates (syntactic, schema, semantic) passed. On failure, similarly note which predicate failed.
- **Replay Behavior:** A given result cannot produce a stronger outcome on replay, because the contract identity fixes its admissibility.
- **Semantic Drift Detection:** By running jobs at varied capacities but with the same contractID, one can detect if different valid outputs were (wrongly) admitted. The patentable claim is the *binding of contract identity to the admission decision logic*.

Implementation evidence: The terminal experiment itself is the evidence: "**the runtime was accidentally implementing: Accept(r) ≈ JSONValid(r), while carrying ResultSchema in identity as though it had consequence**." Thus it failed to enforce the declared contract. The fix is to enforce it by design, which we propose as a concrete invention.

Prior art: Schema-based constrained decoding is known【68†L126-L134】. Tools like Guidance/Azure OpenAI support JSON schema prompts【68†L126-L134】, but they typically enforce only syntactic form. What is novel here is the separation: carrying a schema identity, then *requiring exact schema compliance at admission*. This is reminiscent of API contract systems and provenance (e.g. W3C PROV could record a contract entity), but not exactly implemented at runtime. In-toto signatures bind code and artifact identity, which is similar in spirit but not specialized to generative LLM results. No known patent or paper describes automatically "replaying generator output under a fixed contract ID".

Distinguishing factors: The inventive concept is **identity-bound semantic admission of nondeterministic outputs**. That is, not only using JSON schema for prompt design (which is standard), but using it as an enforced boundary between model and downstream. In other words, "only admit a generated result if it conforms to the **exact same** declared contract that was part of the request identity." This prevents the situation where a model returns a *different* but valid output that violates the spec.

Claim-scope risk: Generic "validate output meets schema" is known. The patentable novelty is the *coupling* of contract to job identity and admission. Claims should emphasize this binding and the fail-closed result.

Recommendation: Merge/Hold (not immediate Tier 1). It's patentable, but prior art (constrained decoding, schema validation) is strong, so focus claims narrowly on the identity-binding mechanism.

Suggested claim (dependent on pipeline claim): “The method of claim [X] further comprising: including a **contract identity** in a job identifier, where the contract identity represents a declared result schema; and requiring that generated output be validated against the schema corresponding to the contract identity before admitting the output as a result.”

Negative control: Re-run the generation at different capacities and show the system rejects output not matching the bound contract. Or explicitly try to feed a mismatched valid JSON and verify it is rejected at the admission boundary.

Comparative Matrix of Candidates

Candidate	Novelty	Prior-Art (ease of find)	Implementation Status	Claim Risk (SME)	Recommendation	Dev Effort
State Admission Engine	★★★★★	Medium (container admission 【59†L47-L52】)	Partial (spec exists, code needed)	Medium (abstract idea)	**File**	High
Transition Validator	★★★★★	Medium (transaction checks)	Partial (conceptual)	Medium	**File**	Med
Admission Pipeline Architecture	★★★★☆	High (semantics known 【12†L354-L362】)	Partial (architecture known)	Low (structure)	**File**	Med
Predicate Algebra & Receipts	★★★★☆	Medium (in-toto, prov)	Minimal (conceptual)	Medium	**File** (dependent)	Low
Reconstitution Firewall	★★★★☆	High (checkpointing)	Conceptual	Medium	Provisional	Med
Consequence Identifiability Gate	★★★☆☆	High (LTL monitors 【24†L13-L20】)	None (new)	High	Provisional	High
Independence (Counterfactual) Assay	★★★☆☆	High (causal inference)	None	High	Hold/Prov	High

Candidate	Novelty	Prior-Art (ease of find)	Implementation Status	Claim Risk (SME)	Recommendation	Dev Effort
Observation Selection (Active Lrn)	★★☆☆☆	Medium (active learning【27†L49-L57】)	None	High	Hold	High
Result-Contract Binding	★★★☆☆	High (schema techniques【68†L126-L134】)	Partial (bug exists)	High	Merge/Hold	Med
Semantic Drift Detection	★★☆☆☆	Medium (no direct analog)	None	High	Hold	High

12. Claim Drafts

Below are sample claim outlines for the top candidates:

- **Claim 1 (Admission Engine, independent):** *"A computer-implemented method comprising:* receiving a candidate state snapshot; applying a typed admission engine with a set of predicates (including at least byte-hash verification and authority-scope validation) to the candidate state; generating a receipt for each predicate; and only if all mandatory predicates pass, converting the candidate state into an authoritative runtime state, else aborting. Each receipt comprises evidence of predicate satisfaction or failure."*

Dependents: claim byte-identity predicate (with hash), authority predicate, time-scoped predicate; fail-closed semantics; use of reception/replay rules in receipt.

- **Claim 2 (Transition Validator, independent):** *"A system for validating state transitions, comprising: a state monitor that identifies distinctions introduced by a proposed state transition; and a validator that determines exactly one lawful disposition for each such distinction from a predefined set (e.g. 'transformed', 'superseded', etc.), where the transition is accepted only if each distinction has exactly one disposition."*

Dependents: definition of dispositions; rule for assignment; record receipt of dispositions; many-to-one mapping of withheld outcomes.

- **Claim 3 (Admission Predicate Algebra, independent):** *"A device that manages admission predicates as first-class objects: each predicate has a fixed type, evaluation procedure, and generates a typed receipt (with evidence and reopening rule). The system records the union of all receipts as an admission record and enforces that no predicate result can be substituted by default."*

This claim would support the receipts subfamily.

(Additional claims would cover identifiability states, lockstep of contract identity, etc.)

13. Summary and Next Steps

In summary, the **Typed State Admission Engine** and the **Transition Validator** are the clearest engineering innovations, with ready traction against implementation and suitable patent framing. The admission engine is essentially a software admission controller with typed predicates【59†L47-L52】, while the validator ensures one-to-one disposition closure. We should file or provisionally protect these first, with dependent claims for receipts and pipeline architecture. The more speculative machinery (identifiability, observation selection) can be prototyped and possibly claimed later as narrower add-ons.

Finally, to **enable** these, we recommend building small prototypes and tests: implement the admission stage in code, add receipt logging, then write unit tests as negative controls (bad snapshot, replay with altered parameters, contract-mismatched output, etc.). The more complex protocols (swap tests, query planning) can be simulated offline or in tests to guide implementation.

This strategy will align with USPTO guidance: we claim *specific technical improvements* (predictable algorithms, state machines, concrete data receipts) rather than abstract policy. For example, Examiner Guidance emphasizes patentable improvements to computer function【17†L166-L175】【18†L67-L74】, and our claims are framed exactly as algorithmic components of a runtime system.

References: We cited authoritative sources on admission control【59†L47-L52】, runtime verification【24†L13-L20】, active learning【27†L49-L57】, and USPTO eligibility guidance【17†L166-L175】【18†L67-L74】 to situate these inventions. Prior art like W3C PROV or in-toto can inform dependent claims. All key novel aspects are traceable to code/terminal observations and are grounded in this concrete system.
